FinSight

TABLE OF CONTENT

1. Overview
2. Functional Requirements
3. Non Functional Requirements
4. Models
5. API endpoints
6. Rate Limiter
7. High Level Design (suggested)

1. OVERVIEW

FinSight is a **RESTful API** built with **Django and Django REST Framework** for managing personal finances. It provides endpoints for **user authentication, role-based authorization, transaction management, and a financial dashboard**. This implements **caching via Redis**, **PostgreSQL as a database**, and a **rate limiter** controlling the number of requests made to an api endpoint.

Tech Stack

- **Language**: Python3
- **Framework**: Django + Django REST Framework
- **Caching**: Redis
- **Database**: PostgreSQL
- **Containerization**: Docker + Docker Compose

2. Functional Requirements

---

1. User and Role Management
   i. Provide a way to manage users and their access levels within the system.
   ii. Roles are defined as:
      a. Viewer: Can only view dashboard data
      b. Analyst: Can view records and access insights
      c. Admin: Can create, update, and manage records and users
   iii. Your backend should support:
      a. Creating and managing users
      b. Assigning roles to users
      c. Managing user status such as active or inactive
      d. Restricting actions based on roles
2. Financial Records Management
   I. Application should support operations such as:
      a. Creating records
      b. Viewing records
      c. Updating records
      d. Deleting records
   II. Filtering records based on criteria such as date, category, or type
3. Dashboard Summary APIs
   I. Provide APIs or backend logic that can return summary level data for a dashboard. Summary includes:
      a. Total income
      b. Total expenses
      c. Net balance
      d. Category wise totals
      e. Recent activity
      f. Monthly or weekly trends
4. Validation and Error Handling
5. Data Persistence
6. Optional requirements
   I. Authentication using tokens or sessions
   II. Rate limiting
   III. API documentation

3. Non - Functional Requirements

1. Available
2. Scalable
3. Low latency

4. Models

1. **User model, authentication and permissions**
    1. This module defines a custom Django user model with role-based access control (RBAC). It includes a UserManager for creating users and superusers, a User model extending AbstractBaseUser, and three permission classes for use with Django REST Framework.

```python
class User(AbstractBaseUser, PermissionsMixin):
    class Role(models.TextChoices):
        VIEWER  = "viewer",  "Viewer"
        ANALYST = "analyst", "Analyst"
        ADMIN   = "admin",   "Admin"

    id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=255)
    email = models.CharField(unique=True)
    role = models.CharField(max_length=10, choices=Role.choices, default=Role.VIEWER)

    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    objects = UserManager()

    USERNAME_FIELD  = "email"
    REQUIRED_FIELDS = ["name"]

    class Meta:
        ordering = ["id"]

    def __str__(self):
        return f"{self.name} - [{self.role}]"

    def is_viewer(self):
        return self.role == self.Role.VIEWER

    def is_analyst(self):
        return self.role == self.Role.ANALYST

    def is_admin_role(self):
        return self.role == self.Role.ADMIN
```

2. Table **User:**

3. Table **UserManager:**
   I. Extends BaseUserManager for custom create_user and create_superuser. Validates email, sets role based on type (Viewer, Analyst, Admin), handles superuser flags.

```python
class UserManager(BaseUserManager):
    def create_user(self, email, name, password=None, **extra_fields):
        if not email:
            raise ValueError("Email is required")

        email = self.normalize_email(email)
        user = self.model(email=email, name=name, **extra_fields)
        user.set_password(password)
        user.save(using=self._db)
        return user


    def create_superuser(self, email, name, password=None, **extra_fields):
        extra_fields.setdefault("is_staff", True)
        extra_fields.setdefault("is_superuser", True)
        extra_fields.setdefault("is_active", True)

        if extra_fields.get("is_staff") is not True:
            raise ValueError("Superuser must have is_staff=True")
        if extra_fields.get("is_superuser") is not True:
            raise ValueError("Superuser must have is_superuser=True")

        return self.create_user(email, name, password, **extra_fields)
```

4. **permissions.py file:**

   a. **Viewer Permission:** Defines base permission for any authenticated user. Requires authentication check: returns True if user is authenticated.

   b. **Analyst Permission:** Allows Analyst or Admin roles only. Checks if user has "Analyst" or "Admin" role after authentication.

   c. **Admin Permission:** Restricts to Admin role exclusively. Returns True only if user role equals "Admin".

2. Transactions model

a. This model depicts a Transaction model for financial operations. It includes transaction types and categories with choices.
b. Transaction Types: **Deposit, Withdrawal**
c. Payment Types: **Payroll, parts_purchase, shop_expense, service_charge**
d. Model Fields
   a. **id:** AutoField primary key.

```python
class IsViewer(BasePermission):
    """
    Viewer, Analyst, Admin — any authenticated user
    """
    message = "Authentication required"

    def has_permission(self, request, view):
        return bool(request.user and request.user.is_authenticated)


class IsAnalyst(BasePermission):
    """
    Analyst and Admin only
    """
    message = "Analyst or Admin access required"

    def has_permission(self, request, view):
        if not (request.user and request.user.is_authenticated):
            return False
        return request.user.role in ("analyst", "admin")


class IsAdminRole(BasePermission):
    """
    Admin only
    """
    message = "Admin access required"

    def has_permission(self, request, view):
        if not (request.user and request.user.is_authenticated):
            return False
        return request.user.role == "admin"
```

   b. **amount:** DecimalField (digits=10, decimals=2, default=0.00).
   c. **category:** CharField (max_length=30, choices from Payment types).
   d. **type:** CharField (max_length=20, choices from Transaction types).
e. Meta Options
   a. Unique ordering by type then category.
   b. Uses verbose_name “Transaction"

5. API ENDPOINTS

---

1. **Accounts**
   a. /api/accounts/register/
   b. /api/accounts/login/
   c. /api/accounts/logout/
2. **Dashboard:** landing page
3. **Transactions**
   a. GET /transactions/list/all/ : list all transactions
   b. POST /transactions/create/: create new transactions
   c. GET /transactions/retrieve/<pk>/: retrieve transaction by id
   d. PUT /transactions/update/<pk>/: update transactions
   e. PATCH /transactions/partial-update/<pk>/: partial update transactions
   f. DELETE /transactions/destroy/<pk>/: delete transactions
   g. GET /transactions/recent/: fetch recent transactions

```python
class Transaction(models.Model):
    class TransactionType(models.TextChoices):
        DEPOSIT = "deposit", "Deposit"
        WITHDRAWN = "withdrawn", "Withdrawn"

    class TransactionCategory(models.TextChoices):
        PAYROLL = "payroll", "Payroll"
        PARTS_PURCHASE = "parts_purchase", "Parts Purchase"
        SHOP_EXPENSE = "shop_expense", "Shop Expense"
        SERVICE_CHARGE = "service_charge", "Service Charge"

    id = models.AutoField(primary_key=True)
    transaction_at = models.DateTimeField(auto_now_add=True)
    amount = models.DecimalField(max_digits=12, decimal_places=2, default=0.00)
    transaction_type = models.CharField(max_length=10, choices=TransactionType.choices)
    category = models.CharField(max_length=20, choices=TransactionCategory.choices)
    description = models.CharField(max_length=255)

    class Meta:
        ordering = ["-transaction_at"]

    def __str__(self):
        return f"{self.transaction_type} - {self.category}"
```

h. GET /transactions/filter-transaction/: filter functions

6. RATE LIMITER

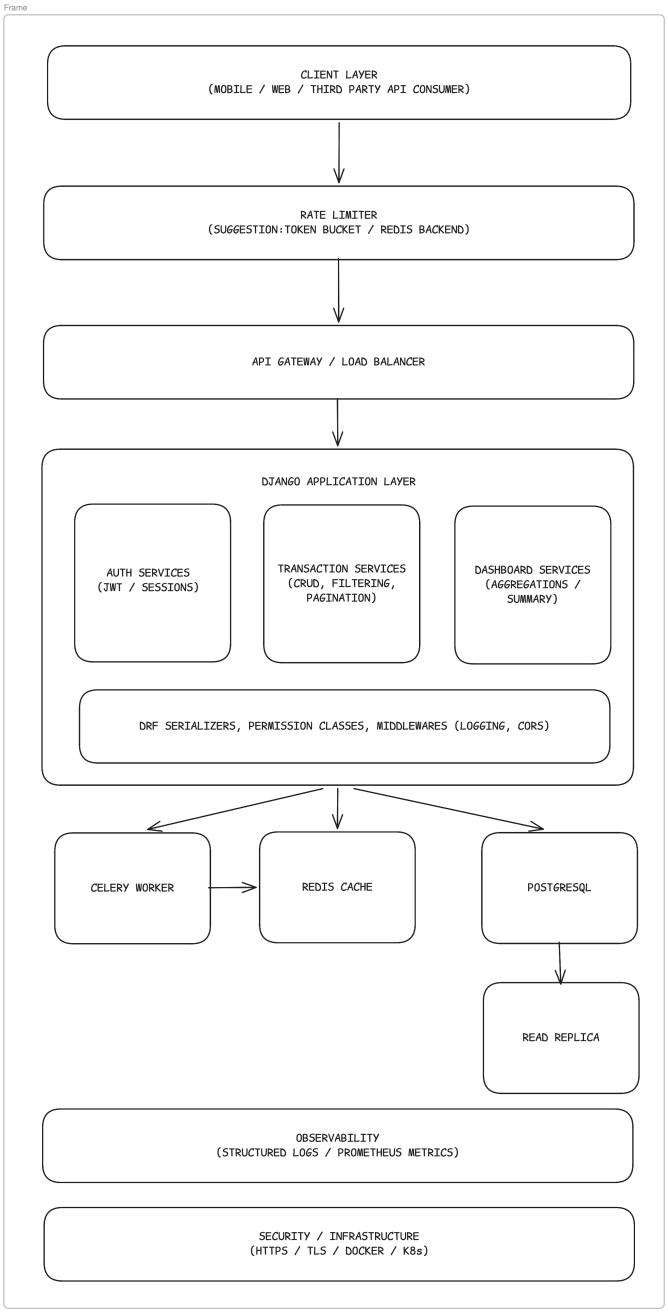
FinSight uses a Redis-based rate limiter implemented as a Python decorator. It tracks the number of requests per endpoint within a time window and blocks excessive requests with a **429 Too Many Requests** response.

```python
@rate_limit(limit=100, window=60, key_func="global")
def my_view(request):
    ...
```

- **How It Works**

- Each incoming request is identified and mapped to a unique Redis key.
- A counter is incremented in Redis for every request.
- If the counter exceeds the allowed limit within the time window, the request is blocked.
- If the request is allowed, rate limit headers are attached to the response.

- **Usage**: Apply the @rate_limit decorator to any view (FBV or CBV):

  - Parameters
    - **limit (int)** — Maximum number of requests allowed in the time window.
    - **window (int)** — Time window in seconds.
    - **key_func (str)** — Identifier string used to namespace the Redis key.

7. HIGH LEVEL DESIGN